

Code templates & reference notes

Track 16.2 – ■ TypeScript for AI Applications · 16-ai-tech-stack/tracks/typescript/code/README.md · 16-ai-tech-stack/tracks/typescript/code/node_modules/@esbuild/darwin-arm64/README.md · 16-ai-tech-stack/tracks/typescript/code/node_modules/@types/node/README.md · 16-ai-tech-stack/tracks/typescript/code/node_modules/@typescript/typescript-darwin-arm64/README.md · 16-ai-tech-stack/tracks/typescript/code/node_modules/esbuild/LICENSE.md · 16-ai-tech-stack/tracks/typescript/code/node_modules/esbuild/README.md · 16-ai-tech-stack/tracks/typescript/code/node_modules/tsx/README.md · 16-ai-tech-stack/tracks/typescript/code/node_modules/typescript/README.md · 16-ai-tech-stack/tracks/typescript/code/node_modules/typescript/vendor/vscode-jsonrpc/README.md · 16-ai-tech-stack/tracks/typescript/code/node_modules/undici-types/README.md · 16-ai-tech-stack/tracks/typescript/code/node_modules/zod/README.md · 16-ai-tech-stack/tracks/typescript/code/node_modules/zod-to-json-schema/README.md · 16-ai-tech-stack/tracks/typescript/code/node_modules/zod-to-json-schema/changelog.md · 16-ai-tech-stack/tracks/typescript/code/node_modules/zod-to-json-schema/contributing.md

Running the examples

Both scripts are **offline**. There is no network call, no API key, and no model provider. The model is simulated — deliberately badly, so you see the failure modes you would otherwise only meet in production.

```
cd 16-ai-tech-stack/tracks/typescript/code
npm install          # zod, zod-to-json-schema, tsx, typescript, @types/node
npx tsx typed-tool-calling.ts
npx tsx streaming-parser.ts
```

Or via the scripts:

```
npm run typecheck    # tsc --noEmit under the strict-plus tsconfig
npm run tools        # typed-tool-calling.ts
npm run stream       # streaming-parser.ts
npm run all           # all three, in order
```

Verified on Node 24 with zod@4, typescript@7, tsx@4. Node 20+ is enough — the only runtime globals used are TextDecoder, TextEncoder, ReadableStream and AbortController, all of which have been stable since Node 18.

What each file is for

File	Teaches
<code>typed-tool-calling.ts</code>	Zod as the single source of truth for a tool: static type, runtime validator, and JSON Schema all derived from one declaration. Then: rejecting a malformed tool call, an invented tool name, and a "5"-instead-of-5 argument, all without throwing. Exhaustiveness checking via <code>assertNever</code> .
<code>streaming-parser.ts</code>	SSE framing over hostile chunk boundaries. Reproduces the split-UTF-8 bug and the dropped-frame bug, then fixes both. Incremental JSON accumulation for streamed tool arguments, mid-stream <code>error</code> events, <code>AbortController</code> , and converting an async iterator into a <code>ReadableStream</code> for a route handler.

`tsconfig.json`

The config is the lesson. `strict: true` is table stakes; the three flags that change how you write code are `noUncheckedIndexedAccess`, `exactOptionalPropertyTypes` and `verbatimModuleSyntax`. See `../notes/02-streaming-runtimes-and-hygiene.md §6`.

If `npm install` is blocked

`streaming-parser.ts` has **zero dependencies** — it will run under `npx tsx` (or `node --experimental-strip-types streaming-parser.ts` on Node 22.6+) with nothing installed. `typed-tool-calling.ts` needs `zod`; the `zod-to-json-schema` section is wrapped in a dynamic `import()` and degrades to a printed explanation if that package is absent.

esbuild

This is the macOS ARM 64-bit binary for esbuild, a JavaScript bundler and minifier. See <https://github.com/evanw/esbuild> for details.

Installation

```
npm install --save @types/node
```

Summary

This package contains type definitions for node (<https://nodejs.org/>).

Details

Files were exported from <https://github.com/DefinitelyTyped/DefinitelyTyped/tree/master/types/node/v24>.

Additional Details

- Last updated: Wed, 09 Sep 2026 18:05:03 GMT

- Dependencies: [undici-types](#)

Credits

These definitions were written by [Microsoft TypeScript](#), [Alberto Schiabel](#), [Andrew Makarov](#), [Benjamin Toueg](#), [David Junger](#), [Mohsen Azimi](#), [Nikita Galkin](#), [Sebastian Silbermann](#), [Wilco Bakker](#), [Marcin Kopacz](#), [Trivikram Kamat](#), [Junxiao Shi](#), [Ilia Baryshnikov](#), [ExE Boss](#), [Piotr Błażejewicz](#), [Anna Henningsen](#), [Victor Perin](#), [NodeJS Contributors](#), [Linus Unnebäck](#), [wafuwafu13](#), [Matteo Collina](#), [Dmitry Semigradsky](#), [René](#), and [Yagiz Nizipli](#).

`@typescript/typescript-darwin-arm64`

This package provides darwin-arm64 support for [typescript](#).

LICENSE

MIT License

Copyright (c) 2020 Evan Wallace

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

esbuild

This is a JavaScript bundler and minifier. See <https://github.com/evanw/esbuild> and the [JavaScript API documentation](#) for details.

README



npm v4.23.15 downloads 299.6M/month

TypeScript Execute (tsx): The easiest way to run TypeScript in Node.js

[Documentation](#)

|

[Getting started →](#)

Love using this project?

 [Buy me coffee](#)

I'm glad you found it so useful! If you're feeling generous, I'd appreciate a cup of coffee. Thanks!

Sponsors-only benefits

 [Join for \\$4](#)

Access the private dev repo for free support, pre-release downloads, and influence project planning!

Already a sponsor? Join the discussion in the [Development repo!](#)

Sponsors

Gold Sponsors



StackBlitz

Silver Sponsors



Afilio



Typed Rocks



WebdriverIO



openclaw

Bronze Sponsors



Anthony Fu



Steve

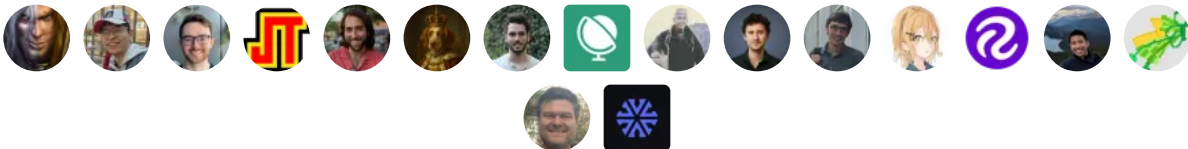


Endear

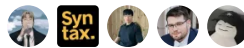


Grey

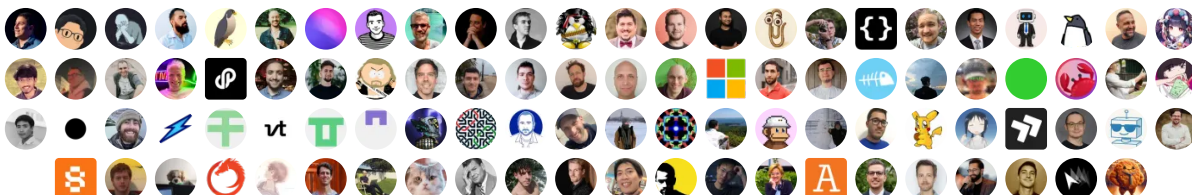
Sponsors



Donors



Past Sponsors



TypeScript



[TypeScript](#) is a language for application-scale JavaScript. TypeScript adds optional types to JavaScript that support tools for large-scale JavaScript applications for any browser, for any host, on any OS. TypeScript compiles to readable, standards-based JavaScript. Try it out at the [playground](#), and stay up to date via [our blog](#) and [Bluesky](#).

Find others who are using TypeScript at [our community page](#).

Installing

For the latest stable version:

```
npm install -D typescript
```

For our nightly builds:

```
npm install -D typescript@next
```

Contribute

There are many ways to [contribute](#) to TypeScript.

- [Submit bugs](#) and help us verify fixes as they are checked in.
- Review the [source code changes](#).
- Engage with other TypeScript users and developers on [Stack Overflow](#).
- Help each other in the [TypeScript Community Discord](#).
- Join the [#typescript](#) discussion on Bluesky.
- [Contribute bug fixes](#).

This project has adopted the [Microsoft Open Source Code of Conduct](#). For more information see the [Code of Conduct FAQ](#) or contact opencode@microsoft.com [16-ai-tech-stack/tracks/typescript/code/node_modules/typescript/mailto:opencode@microsoft.com] with any additional questions or comments.

Documentation

- [TypeScript in 5 minutes](#)
- [Programming handbook](#)
- [Homepage](#)

Roadmap

For details on our planned features and future direction, please refer to our [roadmap](#).

VSCode JSON RPC

npm v9.0.2 downloads 80M/month  Azure Pipelines succeeded

This npm module implements the base messaging protocol spoken between a VSCode language server and a VSCode language client.

The npm module can also be used standalone to establish a [JSON-RPC](#) channel between a client and a server. Below an example how to setup a JSON-RPC connection. First the client side.

```
import * as cp from 'child_process';
import * as rpc from 'vscode-jsonrpc/node';

let childProcess = cp.spawn(...);

// Use stdin and stdout for communication:
let connection = rpc.createMessageConnection(
  new rpc.StreamMessageReader(childProcess.stdout),
  new rpc.StreamMessageWriter(childProcess.stdin));

let notification = new rpc.NotificationType<string, void>('testNotification');

connection.listen();

connection.sendNotification(notification, 'Hello World');
```

The server side looks very symmetrical:

```
import * as rpc from 'vscode-jsonrpc/node';

let connection = rpc.createMessageConnection(
  new rpc.StreamMessageReader(process.stdin),
  new rpc.StreamMessageWriter(process.stdout));

let notification = new rpc.NotificationType<string, void>('testNotification');
connection.onNotification(notification, (param: string) => {
  console.log(param); // This prints Hello World
});

connection.listen();
```

History

For the history please see the [main repository](#)

License

[MIT](#)

undici-types

This package is a dual-publish of the [undici](#) library types. The `undici` package **still contains types**. This package is for users who *only* need undici types (such as for `@types/node`). It is published alongside every release of `undici`, so you can always use the same version.

- [GitHub nodejs/undici](#)
- [Undici Documentation](#)

README



Zod

TypeScript-first schema validation with static type inference
by [@colinhacks](#)



[Docs](#) • [Discord](#) • [X](#) • [Bluesky](#)

[Read the docs →](#)

What is Zod?

Zod is a TypeScript-first validation library. Define a schema and parse some data with it. You'll get back a strongly typed, validated result.

```
import * as z from "zod";

const User = z.object({
  name: z.string(),
});

// some untrusted data...
const input = {
  /* stuff */
};

// the parsed result is validated and type safe!
const data = User.parse(input);

// so you can use it with confidence :)
console.log(data.name);
```

Features

- Zero external dependencies
- Works in Node.js and all modern browsers
- Tiny: 2kb core bundle (gzipped)
- Immutable API: methods return a new instance
- Concise interface
- Works with TypeScript and plain JS
- Built-in JSON Schema conversion
- Extensive ecosystem

Installation

```
npm install zod
```

Basic usage

Before you can do anything else, you need to define a schema. For the purposes of this guide, we'll use a simple object schema.

```
import * as z from "zod";

const Player = z.object({
  username: z.string(),
  xp: z.number(),
});
```

Parsing data

Given any Zod schema, use `.parse` to validate an input. If it's valid, Zod returns a strongly-typed *deep clone* of the input.

```
Player.parse({ username: "billie", xp: 100 });
// => returns { username: "billie", xp: 100 }
```

Note — If your schema uses certain asynchronous APIs like `async` [refinements](#) or [transforms](#), you'll need to use the `.parseAsync()` method instead.

```
const schema = z.string().refine(async (val) => val.length <= 8);

await schema.parseAsync("hello");
// => "hello"
```

AOT compilation

For hot validation paths, `z.compile(schema)` returns a schema clone with an ahead-of-time compiled fast path. Valid inputs take the compiled path; invalid inputs fall back to the regular parser so error reporting stays identical.

Across a 55-schema benchmark the median speedup is **2.4x**, and it scales with how much work the schema does per parse: a large array of objects is $\sim 9x$, a 20-key object $\sim 9x$, a nested object $\sim 4.5x$, while a bare `z.string()` gains nothing — compilation removes per-node dispatch and allocation, and a single `typeof` has none to remove.

```
const CompiledPlayer = z.compile(Player);

CompiledPlayer.parse({ username: "billie", xp: 100 });
```

To enable compilation globally for schemas constructed after import:

```
import "zod/compile"; // place before modules that define schemas
```

Things to know:

- Compilation uses `new Function`. Global mode is automatically disabled when `z.config({ jitless: true })` is set (e.g. CSP environments); calling `z.compile()` directly is an explicit opt-in.
- Schemas with async refinements or transforms can't be compiled, and neither can a few other constructs. That is not an error: `z.compile()` hands the schema back unchanged and it keeps using the regular parser, exactly as global mode leaves it. Pass `{ strict: true }` to throw `ZodCompileAsyncError` / `ZodCompileUnsupportedError` instead.
- On invalid input, refinements and transforms may run twice (fast path, then fallback).
- Deriving a new schema from a compiled one (`.refine()`, `.extend()`, etc.) returns an uncompiled schema — compile the final schema.

See [compile docs](#) for details.

Handling errors

When validation fails, the `.parse()` method will throw a `ZodError` instance with granular information about the validation issues.

```

try {
  Player.parse({ username: 42, xp: "100" });
} catch (err) {
  if (err instanceof z.ZodError) {
    err.issues;
    /* [
      {
        expected: 'string',
        code: 'invalid_type',
        path: [ 'username' ],
        message: 'Invalid input: expected string, received number'
      },
      {
        expected: 'number',
        code: 'invalid_type',
        path: [ 'xp' ],
        message: 'Invalid input: expected number, received string'
      }
    ] */
  }
}

```

To avoid a `try/catch` block, you can use the `.safeParse()` method to get back a plain result object containing either the successfully parsed data or a `ZodError`. The result type is a [discriminated union](#), so you can handle both cases conveniently.

```

const result = Player.safeParse({ username: 42, xp: "100" });
if (!result.success) {
  result.error; // ZodError instance
} else {
  result.data; // { username: string; xp: number }
}

```

Note — If your schema uses certain asynchronous APIs like [async refinements](#) or [transforms](#), you'll need to use the `.safeParseAsync()` method instead.

```

const schema = z.string().refine(async (val) => val.length <= 8);

await schema.safeParseAsync("hello");
// => { success: true; data: "hello" }

```

Inferring types

Zod infers a static type from your schema definitions. You can extract this type with the `z.infer<>` utility and use it however you like.

```
const Player = z.object({
  username: z.string(),
  xp: z.number(),
});

// extract the inferred type
type Player = z.infer<typeof Player>;

// use it in your code
const player: Player = { username: "billie", xp: 100 };
```

In some cases, the input & output types of a schema can diverge. For instance, the `.transform()` API can convert the input from one type to another. In these cases, you can extract the input and output types independently:

```
const mySchema = z.string().transform((val) => val.length);

type MySchemaIn = z.input<typeof mySchema>;
// => string

type MySchemaOut = z.output<typeof mySchema>; // equivalent to z.infer<typeof mySchema>
// number
```

Zod to Json Schema

npm v3.25.2 downloads 44M/week

Notice of deprecation

As of November 2025, this project will no longer be actively maintained. [Zod v4 natively supports generating JSON schemas](#), so I recommend you switch to the new major.

Thank you to all the contributors and sponsors throughout the years! So long, and thanks for all the fish.

Summary

Does what it says on the tin; converts [Zod schemas](#) into [JSON schemas](#)!

- Supports all relevant schema types, basic string, number and array length validations and string patterns.
- Resolves recursive and recurring schemas with internal `$refs`.
- Supports targeting legacy Open API 3.0 specification (3.1 supports regular Json Schema).
- Supports Open AI strict mode schemas (Optional object properties are replaced with required but nullable ones).
- As of v3.25 you can use Zod v4 as a peer-dependency, so long as you still provide v3-schemas.

Looking for the exact opposite? Check out [json-schema-to-zod](#)

Sponsors

As this project will no longer be receiving updates I am no longer accepting new partnerships. Huge shout-out to CodeRabbit for keeping it real to the end <3



Cut code review time & bugs in half
coderabbit.ai

Usage

Basic example

```
import { z } from "zod"; // Or, using v3.25 or v4, "zod/v3"
import { zodToJsonSchema } from "zod-to-json-schema";

const mySchema = z
  .object({
    myString: z.string().min(5),
    myUnion: z.union([z.number(), z.boolean()]),
  })
  .describe("My neat object schema");

const jsonSchema = zodToJsonSchema(mySchema, "mySchema");
```

Expected output

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$ref": "#/definitions/mySchema",
  "definitions": {
    "mySchema": {
      "description": "My neat object schema",
      "type": "object",
      "properties": {
        "myString": {
          "type": "string",
          "minLength": 5
        },
        "myUnion": {
          "type": ["number", "boolean"]
        }
      },
      "additionalProperties": false,
      "required": ["myString", "myUnion"]
    }
  }
}
```

Options

Schema name

You can pass a string as the second parameter of the main `zodToJsonSchema` function. If you do, your schema will end up inside a `definitions` object property on the root and referenced from there. Alternatively, you can pass the name as the `name` property of the options object (see below).

Options object

Instead of the schema name (or nothing), you can pass an options object as the second parameter. The following options are available:

Option	Effect
name?: <i>string</i>	As described above.
nameStrategy?: "ref" "title"	Adds name as "title" meta instead of as a ref as described above
basePath?: string[]	The base path of the root reference builder. Defaults to ["#"].
\$refStrategy?: "root" "relative" "seen" "none"	The reference builder strategy; • "root" resolves \$refs from the root up, ie: "#/definitions/mySchema". • "relative" uses relative JSON pointers. See <i>known issues</i>! • "seen" reuses the output of any "seen" Zod schema. In theory it's a more performant version of "none", but in practice this behaviour can cause issues with nested schemas and has now gotten its own option. • "none" ignores referencing all together, creating a new schema branch even on "seen" schemas. Recursive references defaults to "any", ie <code>{}</code>. Defaults to "root".
effectStrategy?: "input" "any"	The effects output strategy. Defaults to "input". See <i>known issues</i> !
dateStrategy?: "format:date" "format:date-time" "string" "integer"	Date strategy, integer allow to specify in unix-time min and max values. "format:date" creates a string schema with format: "date". "format:date-time" creates a string schema with format: "date-time". "string" is interpreted as "format:date-time". "integer" creates an integer schema with format "unix-time" (unless target "openApi3" is used min max checks are also respected)
emailStrategy?: "format:email" "format:iddn-email" "pattern:zod"	Choose how to handle the email string check. Defaults to "format:email".
base64Strategy?: "format:binary" "contentEncoding:base64" "pattern:zod"	Choose how to handle the base64 string check. Defaults to "contentEncoding:base64" as described here . Note that "format:binary" is not represented in the output type as it's not part of the JSON Schema spec and only intended to be used when targeting OpenAPI 3.0. Later versions of OpenAPI support contentEncoding.
definitionPath?: "definitions" "\$defs"	The name of the definitions property when name is passed. Defaults to "definitions".
target?: "jsonSchema7" "jsonSchema2019-09" "openApi3" "openAi"	Which spec to target. Defaults to "jsonSchema7"

Option	Effect
strictUnions?: boolean	Scrubs unions of any-like json schemas, like <code>{}</code> or <code>true</code> . Multiple zod types may result in these out of necessity, such as <code>z.instanceof()</code>
definitions?: Record	See separate section below
errorMessages?: boolean	Include custom error messages created via chained function checks for supported zod types. See section below
markdownDescription?: boolean	Copies the <code>description</code> meta to <code>markdownDescription</code>
patternStrategy? : "escape" "preserve"	The Zod string validations <code>.includes()</code> , <code>.startsWith()</code> , and <code>.endsWith()</code> must be converted to regex to be compatible with JSON Schema's <code>pattern</code> . For safety, all non-alphanumeric characters are escaped by default (consider <code>z.string().includes(".")</code>), but this can occasionally cause problems with Unicode-flagged regex parsers. Use <code>preserve</code> to prevent this escaping behaviour and preserve the exact string written, even if it results in an inaccurate regex.
applyRegexFlag s?: boolean	JSON Schema's <code>pattern</code> doesn't support RegExp flags, but Zod's <code>z.string().regex()</code> does. When this option is true (default false), a best-effort is made to transform regexes into a flag-independent form (e.g. <code>/x/i => /[xX]/</code>). Supported flags: <code>i</code> (basic Latin only), <code>m</code> , <code>s</code> .
pipeStrategy?: "all" "input" "output"	Decide which types should be included when using <code>z.pipe</code> , for example <code>z.string().pipe(z.number())</code> would return both <code>string</code> and <code>number</code> by default, only <code>string</code> for "input" and only <code>number</code> for "output".
removeAdditionalStrategy?: "passthrough" "strict"	Decide when <code>additionalProperties</code> should be allowed. See the section on additional properties for details.
allowedAdditionalProperties?: true undefined	What value to give <code>additionalProperties</code> when allowed. See the section on additional properties for details.
rejectedAdditionalProperties?: false undefined	What value to give <code>additionalProperties</code> when rejected. See the section on additional properties for details.
override?: callback	See section
postProcess?: callback	See section
openAiAnyTypeName?: string	Decides the name of a Json schema used to allow semi-arbitrary values in Open AI structured output. If any value in the Zod-schema resolves to any "any"-type schema it will reference a definition of this name. If no such definition is provided a default Json schema will be used. Defaults to "OpenAiAnyType"

Definitions

The definitions option lets you manually add recurring schemas into definitions for cleaner outputs. It's fully compatible with named schemas, changed definitions path and base path. Here's a simple example:

```
const myRecurringSchema = z.string();
const myObjectSchema = z.object({ a: myRecurringSchema, b: myRecurringSchema });

const myJsonSchema = zodToJsonSchema(myObjectSchema, {
  definitions: { myRecurringSchema },
});
```

Result

```
{
  "type": "object",
  "properties": {
    "a": {
      "$ref": "#/definitions/myRecurringSchema"
    },
    "b": {
      "$ref": "#/definitions/myRecurringSchema"
    }
  },
  "definitions": {
    "myRecurringSchema": {
      "type": "string"
    }
  }
}
```

Error Messages

This feature allows optionally including error messages created via chained function calls for supported zod types:

```
// string schema with additional chained function call checks
const EmailSchema = z.string().email("Invalid email").min(5, "Too short");

const jsonSchema = zodToJsonSchema(EmailSchema, { errorMessages: true });
```

Result

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "string",
  "format": "email",
  "minLength": 5,
  "errorMessage": {
    "format": "Invalid email",
    "minLength": "Too short"
  }
}
```

This allows for field specific, validation step specific error messages which can be useful for building forms and such. This format is accepted by `react-hook-form`'s ajv resolver (and therefor `ajv-errors` which it uses under the hood). Note that if using AJV with this format will require `enabling ajv-errors` as vanilla AJV does not accept this format by default.

Custom Error Message Support

- `ZodString`
- `regex`
- `min`, `max`
- `email`, `cuid`, `uuid`, `url`
- `endsWith`, `startsWith`
- `ZodNumber`
- `min`, `max`, `lt`, `lte`, `gt`, `gte`,
- `int`
- `multipleOf`
- `ZodSet`
- `min`, `max`
- `ZodArray`
- `min`, `max`

Additional properties

By default, Zod removes undeclared properties when parsing object schemas. In order to replicate the expected output of this behaviour, the default for behaviour of `zodToJsonSchema` is to set `"additionalProperties"` to `false` (although the correctness of this can be debated). If you wish to allow undeclared properties you can either:

- Set `removeAdditionalStrategy` to `"strict"`. This will allow additional properties for any object schema that is not declared with `.strict()`.
- Leave `removeAdditionalStrategy` set to its default value of `"passthrough"`, and add `.passthrough()` to your object schema.

Removing the `additionalProperties` keyword using the `allowedAdditionalProperties` and/or `rejectedAdditionalProperties` options.

Some schema definitions (like Google's Gen AI API for instance) does not allow the `additionalProperties` keyword at all. Luckily the JSON Schema spec allows for this: leaving the keyword undefined *should* have the same effect as setting it to true (as per usual YMMV). To enable this behaviour, set the option `allowedAdditionalProperties` to `undefined`.

To exclude the keyword even when additional properties are *not* allowed, set the `rejectedAdditionalProperties` to `undefined` as well.

Heads up ⚠️: Both of these options will be ignored if your schema is declared with `.catchall(...)` as the provided schema will be used instead (if valid).

Expected outputs

<code>z.object({})</code> + option	"additionalProperties" value
<code>.strip()</code> (default)	false if strategy is "passthrough", true if "strict"
<code>.passthrough()</code>	true
<code>.strict()</code>	false
<code>.catchall(z.string())</code>	{ "type": "string" }
<code>.catchall(z.function())</code>	undefined (function schemas are not currently parseable)

Substitute `true` and `false` for `undefined` according to `allowedAdditionalProperties` and/or `rejectedAdditionalProperties` respectively.

override

This options takes a callback receiving a Zod schema definition, the current reference object (containing the current ref path and other options), an argument containing information about whether or not the schema has been encountered before, and a `forceResolution` argument.

Important: if you don't want to override the current item you have to return the `ignoreOverride` symbol exported from the index. This is because `undefined` is a valid option to return when you want the property to be excluded from the resulting JSON schema.

```
import zodToJsonSchema, { ignoreOverride } from "zod-to-json-schema";

zodToJsonSchema(
  z.object({
    ignoreThis: z.string(),
    overrideThis: z.string(),
    removeThis: z.string(),
  }),
  {
    override: (def, refs) => {
      const path = refs.currentPath.join("/");

      if (path === "#/properties/overrideThis") {
        return {
          type: "integer",
        };
      }

      if (path === "#/properties/removeThis") {
        return undefined;
      }

      // Important! Do not return `undefined` or void unless you want to remove the property from the
      // resulting schema completely.
      return ignoreOverride;
    },
  },
);
```

Expected output:

```
{
  "type": "object",
  "required": ["ignoreThis", "overrideThis"],
  "properties": {
    "ignoreThis": {
      "type": "string"
    },
    "overrideThis": {
      "type": "integer"
    }
  },
  "additionalProperties": false
}
```

postProcess

Besides receiving all arguments of the `override` callback, the `postProcess` callback also receives the generated schema. It should always return a JSON Schema, or `undefined` if you wish to filter it out. Unlike the `override` callback you do not have to return `ignoreOverride` if you are happy with the produced schema; simply return it unchanged.

```

import zodToJsonSchema, { PostProcessCallback } from "zod-to-json-schema";

// Define the callback to be used to process the output using the PostProcessCallback type:
const postProcess: PostProcessCallback = (
  // The original output produced by the package itself:
  jsonSchema,
  // The ZodSchema def used to produce the original schema:
  def,
  // The refs object containing the current path, passed options, etc.
  refs,
) => {
  if (!jsonSchema) {
    return jsonSchema;
  }

  // Try to expand description as JSON meta:
  if (jsonSchema.description) {
    try {
      jsonSchema = {
        ...jsonSchema,
        ...JSON.parse(jsonSchema.description),
      };
    } catch {}
  }

  // Make all numbers nullable:
  if ("type" in jsonSchema! && jsonSchema.type === "number") {
    jsonSchema.type = ["number", "null"];
  }

  // Add the refs path, just because
  (jsonSchema as any).path = refs.currentPath;

  return jsonSchema;
};

const jsonSchema = zodToJsonSchema(zodSchema, { postProcess });

```

Using `postProcess` for including examples and other meta

Adding support for examples and other JSON Schema meta keys are among the most commonly requested features for this project. Unfortunately the current Zod major (3) has pretty anemic support for this, so some userland hacking is required. Since this is such a common usecase I've included a helper function that simply tries to parse any description as JSON and expand it into the resulting schema.

Simply stringify whatever you want added to the output schema as the description, then import and use `jsonDescription` as the `postProcess` option:

```
import zodToJsonSchema, { jsonDescription } from "zod-to-json-schema";

const zodSchema = z.string().describe(
  JSON.stringify({
    title: "My string",
    description: "My description",
    examples: ["Foo", "Bar"],
    whatever: 123,
  }),
);

const jsonSchema = zodToJsonSchema(zodSchema, {
  postProcess: jsonDescription,
});
```

Expected output:

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "string",
  "title": "My string",
  "description": "My description",
  "examples": ["Foo", "Bar"],
  "whatever": 123
}
```

Known issues

- The OpenAI target should be considered experimental for now, as some combination of options may break the compatibility.
- When using `.transform`, the return type is inferred from the supplied function. In other words, there is no schema for the return type, and there is no way to convert it in runtime. Currently the JSON schema will therefore reflect the input side of the Zod schema and not necessarily the output (the latter aka. `z.infer`). If this causes problems with your schema, consider using the effectStrategy "any", which will allow any type of output.
- JSON Schemas does not support any other key type than strings for objects. When using `z.record` with any other key type, this will be ignored. An exception to this rule is `z.enum` as is supported since 3.11.3
- Relative JSON pointers, while published alongside [JSON schema draft 2020-12](#), is not technically a part of it. Currently, most resolvers do not handle them at all.
- Since v3, the Object parser uses `.isOptional()` to check if a property should be included in `required` or not. This has the potentially dangerous behavior of calling `.safeParse` with `undefined`. To work around this, make sure your `preprocess` and other effects callbacks are pure and not liable to throw errors. An issue has been logged in the Zod repo and can be [tracked here](#).
- JSON Schema version 2020-12 is not yet officially supported. However, you should be able to use this library to obtain a compatible schema for the 2020-12 format just by changing the returned schema's `$schema` field to: `"https://json-schema.org/draft/2020-12/schema#"`

Versioning

This package *does not* follow semantic versioning. The major and minor versions of this package instead reflects feature parity with the [Zod package](#).

I will do my best to keep API-breaking changes to an absolute minimum, but new features may appear as "patches", such as introducing the options pattern in 3.9.1.

While v3.25 supports Zod v4 as a peer-dependency, it does *not* mean it supports v4 schemas. Use `import { z } from "zod/v3"` and refer to the Zod documentation.

Changelog

<https://github.com/StefanTerdell/zod-to-json-schema/blob/master/changelog.md>

Changelog

Version	Change
3.25.2	Bumps the peer dependency of Zod 3 to 3.25.28 - Versions before patch 13 caused OOM issues and versions between that and 28 removed the /v3 import alias.
3.25.1	Fixes large install size due to accidental inclusion of test files. Thanks, Felix Mosheev !
3.25.0	Adds support for v3.25 and v3 through v4 (<code>import { z } from "zod/v3"</code>). Big thank you to both Andrey Gubanovs and especially to Faiz Hernawan Abdillah , whose more minimal implementation was merged. This will likely be the final release of <code>zod-to-json-schema</code> , as v4 now supports JSON schema natively.
3.24.6	Removed use of <code>instanceOf</code> to check for optional properties as differing package versions could produce intermittent bugs. Added <code>OpenAiAnyType</code> to work around their schema restrictions.
3.24.5	Update <code>.npmignore</code> to drop 2 mb of test files. Thanks Misha Kaletsky !
3.24.4	Added options to set the value of <code>additionalProperties</code> in objects and record
3.24.3	Adds <code>postProcess</code> callback option
3.24.2	Restructured internals to remove circular dependencies which apparently might cause some build systems to whine a bit. Big thanks to V�ctor Hern�ndez for the fix.
3.24.1	Adds OpenAI target
3.24.0	Implements new string checks (<code>jwt</code> , <code>base64url</code> , <code>cidr ipv4/v6</code>), matching the new Zod version
3.23.5	Module import hotfix by Enzo Monjard�n . Thanks!
3.23.4	Fixes branded regex property names and a weird edgecase in arrays. Thanks to Isaiah Marc Sanchez and Mitchell Merry !
3.23.3	More tests (Thanks Brett Zamir !), removed dead code
3.23.2	Lazily loads Emoji regex to avoid incompatibility with some environments. Thanks Jacob Lee !
3.23.1	Best-effort RegEx flag support by Spappz ! Some minor fixes and additions, such as the <code>title</code> option.
3.23.0	Adds support for <code>base64</code> , <code>date</code> , <code>time</code> , <code>duration</code> and <code>nanoid</code> string validations. A warm welcome and a big thanks to Colin , the creator of Zod, joining in as a contributor :)
3.22.5	Adds new <code>z.date()</code> parsing options and override callback
3.22.4	Adds fix for nullable references in OpenAPI mode

Version	Change
3.22.3	Adjust root path from "#/" to "#" according to RFC 6901
3.22.2	Adds "output" pipe strategy
3.22.1	Fixes broken imports when using some bundlers
3.22.0	Support readonly. Export both CJS and ESM. Export everything from index. Alternative map parser. Improved pattern handling and updated sources.
3.21.4	Fixes missing support for exact array length
3.21.3	Fixes issue #77 (Reference path to nullable schemas in Open-API mode)
3.21.2	Adds "integer" type Date output to support min/max checks, markdownDescription option, fixes "none" refStrategy by adding "seen" and adds an option to use "pattern" with Zods' email enum instead of "format".
3.21.1	New target (2019-09) along with improved intersection schemas, improved mutual recursion references in definitions, descriptions respected in union parser and not removed in collapsed
3.21.0	Added new string validations (ip, emoji, etc) and BigInt checks to support Zod 3.21
3.20.5	Added uniqueItems to Set and an option to disregard pipe schemas
3.20.4	Bugfixes and improved record parsing for openApi3
3.20.3	Added Cuid2 support introduced in Zod 3.20.3
3.20.2	Reintroduced conditional simplified return-type for when target is OpenAPI 3
3.20.1	Fixed inconsistent casing in imports
3.20.0	Adds support for Zod 3.20 with catch and pipe parser as well as new string validations. Refactored Ref handling; adding definitions no longer considered experimental. Main API function refactored and simplified; output type less defined but a lot easier to maintain. Doubt anyone will miss it. <i>Narrator: Someone did in fact miss it</i>
3.19.4	Adds custom error message support
3.19.3	Mark <code>definitions</code> as experimental in the readme
3.19.2	Added <code>definitions</code> option

Versi on	Change
3.19. 1	Strict unions fix
3.19. 0	No new features added in Zod, parity bump
3.18. 2	Fixes support for native enums
3.18. 1	Add strictUnions options
3.18. 0	Added support for branded types
3.17. 2	Fix for reference paths when supplying name option string.
3.17. 1	Added startsWith and endsWith string checks. Merge multiple pattern checks into allOf array.
3.17. 0	Added switch case handler for new trim "check". No changes to functionality.
3.15. x - 3.16. x	Skipped: Did not change the Zod API in any way relevant for this package.
3.14. 1	Dependabot security updates
3.14. 0	Moves Zod into peerDependencies. Supports checks for sets, NaN-type (sort of), discriminated union type and standalone optional properties (as unions with undefined)
3.12. x - 3.13. x	Skipped
3.11. 3	Joins unions of enums into single enum and allows enums as keys of records
3.11. 2	Adds option to target Open API 3 spec (paths) instead of Json Schema 7.
3.11. 1	Performance boost when using \$refStrategy none and internal improvements.
3.11. 0	Added description support introduced in Zod 3.11.5
3.10. x	Skipped: Minor 10 did not change the Zod API
3.9.5	Type bug fix: used dev dependency types in package

Version	Change
3.9.4	Path bug fix and test case when using optional definitions path
3.9.3	Added option to change definition property name to \$defs
3.9.2	Added option to handle transform results as any instead of relying on their input schema.
3.9.1	Refactored the way reference pointers are passed around and added options pattern to main function without braking backwards compatibility! You can now add a base path, change the reference strategy (or opt out), and still set the schema name inside the options object or outside as before.
3.9.0	Added support for multipleOf number validaion, .rest() schemas for tuples and key validation for records (only compatible with string keys due to JSON Schema limitation).
3.7.x - 3.8.x	Skipped to reach functional parity with Zod versioning.
3.6.1	Realised intersection had another potential ref pathing bug. Fixed.
3.6.0	Added support for default & effects (refine). Broke out changelog.md
3.5.0	Added support for CUID string validation
3.4.3	Fixed \$ref pathing for catchall and intersection. Additional tests and code structure fixes.
3.4.2	Fixed broken intersection parser (Thanks Noah2610!)
3.4.1	Fixed pathing bug for nullable items.
3.4.0	Added support for z.lazy()
3.3.0	Added support for catchall (additionalProperties schema on objects). Rebuilt object parser.
3.2.0	Added support for Map and Set as seen by their most common JSON definitions. Beware no standardized definition seem to exist and JSON.parse doesn't handle either natively. Their implementations here are based on the spread approach. Also further simplified intersection definition to just allOf.
3.1.0	String patterns finally supported! Fixed bugs include broken external type, unsafe nullable parsing, bad intersection implementation, and missing support for passthrough keys in objects.
3.0.3	Fixed array deep pathing bug (path contained <code>array</code> instead of <code>items</code>)
3.0.2	Fixed broken type usage (NonEmptyArrayDefinition was removed from Zod)
3.0.1	Fixed a typo in the readme
3.0.0	Compatible with Zod 3.2.0. Huge props to Mr Hammad Asif for his work on this.
0.6.2	Hotfix for undefined object properties. Could crash the parser when using Pick
0.6.1	Fixed bug in union pathing. <code>\$Ref</code> was missing <code>/anyOf</code>
0.6.0	Moved <code>@types/json-schema</code> and <code>typescript</code> to dev dependencies. <code>@types/json-schema</code> is now only used for the test suites. Using <code>strict: true</code> in ts config.
0.5.1	First working release with all relevant Zod types present with most validations (except for string patterns due to Zod not exposing the source regexp pattern for those).

Version	Change
< 0.5.1	Deprecated due to broken package structure. Please be patient, I eat crayons.

Contributing

Hey, thanks for wanting to contribute.

Before you open a PR, make sure to open an issue and discuss the problem you want to solve. I will not consider PRs without issues.

I use [gitmoji](#) for my commit messages because I think it's fun. I encourage you to do the same, but won't enforce it.

I check PRs and issues very rarely so please be patient.